

Обектно ориентирано програмиране – Полиморфизъм. Множествено наследяване.

Ивайло Андреев + Claude Sonnet 4.6

25 май 2026

Съдържание

1	Полиморфизъм – общ преглед	2
2	Виртуални функции и подтипов полиморфизъм. Динамично свързване.	2
2.1	Статично срещу динамично свързване	2
2.2	Виртуални функции – правила и спецификатори	3
2.3	Пример: динамично свързване при дълбока йерархия	3
2.4	Абстрактни класове и чисто виртуални функции	5
3	Масиви от обекти и от указатели към обекти	5
3.1	Масив от обекти (на стека)	6
3.2	Динамичен масив от обекти (на heap-a)	6
3.3	Масив от указатели към обекти	6
3.4	Проблемът с нарязването (Object Slicing) и полиморфните контейнери	6
4	Параметричен полиморфизъм. Шаблони на функция и на клас.	7
4.1	Шаблони на функция	7
4.2	Шаблони на клас	8
4.3	Нетипови параметри на шаблон	8
4.4	Специализация на шаблон	9
4.5	Ограничение: дефиниции в хедър файл	9
5	Множествено наследяване	9
5.1	Диамантен проблем (Diamond Problem)	10
5.2	Виртуално наследяване – решение на диамантения проблем	10
5.3	Конструктори при виртуално наследяване	10

1 Полиморфизъм – общ преглед

Полиморфизмът (от гр. „много форми“) е принципът, при който един и същ интерфейс се реализира по различен начин в зависимост от типа на обекта. В C++ съществуват два вида:

- **Статичен (compile-time) полиморфизъм:** изборът на функция се прави по време на компилация. Реализира се чрез претоварване на функции/оператори и **шаблони (templates)**.
- **Динамичен (run-time) / Подтипов полиморфизъм:** изборът се прави по време на изпълнение. Реализира се чрез **наследяване и виртуални функции**.

2 Виртуални функции и подтипов полиморфизъм. Динамично свързване.

2.1 Статично срещу динамично свързване

Без ключовата дума `virtual` компилаторът определя коя функция да извика единствено на базата на **статичния тип** на указателя/референцията (ранно свързване). С `virtual` – на базата на **реалния тип** на обекта по време на изпълнение (късно свързване).

```
1 class Base {
2 public:
3     void f() const { std::cout << "Base::f()\n"; }           // non-virtual
4     virtual void g() const { std::cout << "Base::g()\n"; } // virtual
5 };
6
7 class Derived : public Base {
8 public:
9     void f() const { std::cout << "Derived::f()\n"; }         // hides
10    Base::f
11    void g() const override { std::cout << "Derived::g()\n"; } //
12    overrides Base::g
13 };
14
15 int main() {
16     Derived d;
17     Base* p = &d;
18
19     p->f(); // Base::f()      -- static binding (non-virtual)
20     p->g(); // Derived::g()  -- dynamic binding (virtual)
21 }
```

2.2 Виртуални функции – правила и спецификатори

- **virtual:** Декларира функцията за виртуална в базовия клас. Всички класове-наследници наследяват тази виртуалност автоматично.
- **override:** Изрично указва, че функцията предефинира виртуална функция от базовия клас. Компиляторът проверява – предпазва от типо-грешки в сигнатурата.
- **final:** Забранява по-нататъшно предефиниране на функцията (или наследяване от класа).
- **Виртуален деструктор:** Задължителен в базов клас, ако обектите ще се изтриват чрез указател към базовия клас. Без него `delete p` извиква само деструктора на базовия клас – **изтичане на памет** или неопределено поведение.

```
1 class Shape {
2 public:
3     virtual void draw() const { std::cout << "Drawing shape\n"; }
4     virtual ~Shape() = default;           // virtual destructor -- mandatory!
5 };
6
7 class Circle : public Shape {
8 public:
9     void draw() const override final { // override + forbid further
10         std::cout << "Drawing circle\n";
11     }
12 };
```

Ограничения: Виртуалните функции не могат да бъдат **static** (нямат **this** указател). Конструкторите не могат да бъдат виртуални.

2.3 Пример: динамично свързване при дълбока йерархия

Следният пример илюстрира точно коя функция се извиква при различни комбинации от йерархия и `override`-ване:

```
1 class Base {
2 public:
3     virtual void f() const { std::cout << "Base::f()\n"; }
4     virtual void g() const { std::cout << "Base::g()\n"; }
5     void nonVirtual() const { std::cout << "Base::nonVirtual()\n"; }
6 };
7
8 class FirstDerived : public Base {
9 public:
10     void f() const override { std::cout << "FirstDerived::f()\n"; }
11     void g() const override { std::cout << "FirstDerived::g()\n"; }
12     virtual void h() const { std::cout << "FirstDerived::h()\n"; } //
13     new virtual
```

```

13 };
14
15 class SecondDerived : public FirstDerived {
16 public:
17     void f() const override { std::cout << "SecondDerived::f()\n"; }
18     // g() NOT overridden -- inherited from FirstDerived
19 };
20
21 class ThirdDerived : public SecondDerived {
22 public:
23     void h() const override { std::cout << "ThirdDerived::h()\n"; }
24     // f() NOT overridden -- inherited from SecondDerived
25 };

1 int main() {
2     Base          baseObj;
3     FirstDerived  firstObj;
4     SecondDerived secondObj;
5     ThirdDerived  thirdObj;
6
7     Base* p = nullptr;
8
9     p = &baseObj;
10    p->f();           // Base::f()
11    p->g();           // Base::g()
12    p->nonVirtual(); // Base::nonVirtual() (static -- always Base)
13
14    p = &firstObj;
15    p->f();           // FirstDerived::f()
16    p->g();           // FirstDerived::g()
17
18    p = &secondObj;
19    p->f();           // SecondDerived::f()
20    p->g();           // FirstDerived::g() -- not overridden in
                       SecondDerived!
21
22    p = &thirdObj;
23    p->f();           // SecondDerived::f() -- not overridden in
                       ThirdDerived
24    p->g();           // FirstDerived::g() -- not overridden anywhere
                       below First
25
26    // h() is NOT in Base's interface -- cannot call through Base*
27    // Need FirstDerived* to access h():
28    FirstDerived* fp = &thirdObj;
29    fp->h();          // ThirdDerived::h() -- dynamic dispatch
30 }

```

Ключовото наблюдение: виртуалната функция **продължава да се търси нагоре по йерархията** до намерения override. Ако `SecondDerived` не предефинира `g()`, но `FirstDerived` го е направил – `FirstDerived::g()` се извиква. Невиртуалните методи (`nonVirtual()`) винаги се свързват статично с типа на указателя.

2.4 Абстрактни класове и чисто виртуални функции

Чисто виртуална функция се декларира с `= 0` и задължава всеки конкретен наследник да я предефинира. Клас с поне една чисто виртуална функция е **абстрактен** – от него не могат да се създават обекти директно. Служи като интерфейс.

```
1 class Animal {
2 public:
3     virtual void speak() const = 0;    // pure virtual -- no body here
4     virtual ~Animal() = default;
5 };
6
7 class Dog : public Animal {
8 public:
9     void speak() const override { std::cout << "Woof!\n"; }
10 };
11
12 class Cat : public Animal {
13 public:
14     void speak() const override { std::cout << "Meow!\n"; }
15 };
16
17 // Animal a; // ERROR: cannot instantiate abstract class
18 Animal* a = new Dog();
19 a->speak();    // Woof! -- dynamic dispatch
20 delete a;
```

Чисто виртуална функция **може да има тяло**, дефинирано извън класа. Наследниците могат да го извикват явно чрез `Base::f()`. Класът остава абстрактен – наследниците пак трябва да я предефинират.

```
1 void Animal::speak() const {                // body of pure virtual
2     std::cout << "... (silence)...\n";
3 }
4
5 class Parrot : public Animal {
6 public:
7     void speak() const override {
8         Animal::speak();                    // call base body explicitly
9         std::cout << "Polly wants a cracker!\n";
10    }
11 };
```

3 Масиви от обекти и от указатели към обекти

При работа с наследяване се срещат четири основни варианта на деклариране:

```
1 class Base { public: virtual void f(); virtual ~Base() = default; };
2 class Derived : public Base { public: void f() override; };
3
4 Base    arr[10];    // array of 10 Base objects by value -- no
                    polymorphism
```

```

5 Derived arr[10];    // array of 10 Derived objects by value -- no
   polymorphism
6 Base*   arr;        // pointer to Base -- supports polymorphism (can
   point to Derived)
7 Derived* arr;        // pointer to Derived -- Derived objects only

```

Стойностните масиви (`Base arr[]` и `Derived arr[]`) не поддържат полиморфизъм – при съхранение на произведен обект в Base слот настъпва **object slicing** (виж 3.4). Само масивите от указатели (`Base* arr[]`) позволяват динамично свързване.

3.1 Масив от обекти (на стека)

```

1 class Car { /* ... */ };
2
3 Car arr[5];    // calls default constructor 5 times
4               // destructors called automatically when arr goes out of
   scope

```

Характеристики: Обектите са плътно в паметта. Всички елементи се конструират с един и същ конструктор (по подразбиране). Размерът е фиксиран при компилация.

3.2 Динамичен масив от обекти (на heap-а)

```

1 Car* arr = new Car[5];    // calls default constructor 5 times
2 delete[] arr;             // calls destructor 5 times, then frees memory
3                           // delete[] -- NOT delete!

```

Размерът може да се определи по време на изпълнение. Изисква `delete[]` – `delete` (без `[]`) е неопределено поведение.

3.3 Масив от указатели към обекти

```

1 Car* arr[5];           // array of 5 pointers (uninitialized)
2 arr[0] = new Car(42);   // each element independently constructed
3 arr[1] = new Car(7);
4 // ...
5 for (int i = 0; i < 5; i++) delete arr[i]; // each element independently
   destroyed

```

Предимства: всеки елемент може да е конструиран с различни аргументи; позиция може да бъде `nullptr`; позволява полиморфизъм.

3.4 Проблемът с нарязването (Object Slicing) и полиморфните контейнери

При съхранение на произведен обект **по стойност** в базов тип се губят допълнителните полета и виртуалното поведение – това се нарича **object slicing**:

```

1 class Base {
2 public:
3     virtual std::string name() const { return "Base"; }
4     virtual ~Base() = default;
5 };
6
7 class Derived : public Base {
8 public:
9     std::string name() const override { return "Derived"; }
10    int extra = 42; // extra data, lost on slicing
11 };
12
13 // Problem: storing by value causes slicing
14 Derived d;
15 Base b = d; // SLICING: only the Base part is copied
16 b.name(); // "Base" -- NOT "Derived"!

```

Решението е масив от указатели към базовия клас – полиморфен контейнер:

```

1 Base* arr[2];
2 arr[0] = new Base();
3 arr[1] = new Derived();
4
5 for (int i = 0; i < 2; i++) {
6     std::cout << arr[i]->name() << "\n"; // "Base", then "Derived" --
7     delete arr[i]; // correct because ~Base() is
8 }

```

Задължително: Деструкторът на базовия клас трябва да е `virtual`, за да се извикат правилните деструктори на наследниците при `delete arr[i]`.

4 Параметричен полиморфизъм. Шаблони на функция и на клас.

Параметричният полиморфизъм се реализира чрез **шаблони (templates)** – механизъм за писане на обобщен код, работещ с произволен тип. Типът е параметър, подаден при компилация. Компиляторът **инстанцира** (генерира конкретен код) за всеки използван тип.

4.1 Шаблони на функция

```

1 template <typename T>
2 T maxOf(const T& a, const T& b) {
3     return (a > b) ? a : b;
4 }
5
6 int main() {
7     std::cout << maxOf<int>(3, 7); // explicit: T = int

```

```

8     std::cout << maxOf(3.14, 2.71);      // deduced: T = double (type
deduction)
9     std::cout << maxOf(std::string("a"), std::string("b")); // T = string
10 }

```

Компиляторът генерира отделна функция за всеки тип T – $\text{maxOf}\langle\text{int}\rangle$, $\text{maxOf}\langle\text{double}\rangle$ и т.н. Ако типът не поддържа $>$, се получава грешка при компилация.

Може да се използва `class` вместо `typename` – двете са еквивалентни в този контекст.

4.2 Шаблони на клас

```

1 template <typename T>
2 class Array {
3 public:
4     Array(int n) : size_(n), data_(new T[n]) { }
5     ~Array() { delete[] data_; }
6     T& operator[](int i) { return data_[i]; }
7     int size() const { return size_; }
8 private:
9     int size_;
10    T* data_;
11 };
12
13 int main() {
14     Array<int> nums(5);
15     nums[0] = 10;
16     std::cout << nums.size(); // 5
17
18     Array<std::string> words(3);
19     words[0] = "hello";
20 }

```

4.3 Нетипови параметри на шаблон

`Array<T>` от предишната секция използва динамична памет (heap). Параметрите на шаблона могат да бъдат и конкретни стойности (**нетипови параметри**), което позволява размерът да е известен при компилация – тогава масивът е на стека, без heap заделяне. `Array<T, N>` по-долу е **различна дефиниция**, не предефиниране на същата.

Параметрите на шаблон могат да бъдат и конкретни стойности (не само типове):

```

1 template <typename T, int N>
2 class Array {
3     T data[N];    // fixed-size array, size known at compile time
4 public:
5     int size() const { return N; }
6     T& operator[](int i) { return data[i]; }
7 };
8
9 Array<double, 10> v; // array of 10 doubles, no heap allocation

```


4.4 Специализация на шаблон

Можем да дефинираме специално поведение за конкретен тип:

```
1 template <typename T>
2 T sum(T a, T b) { return a + b; }    // general
3
4 template <>
5 const char* sum(const char* a, const char* b) { // specialization for
6     char*
7     // custom string concatenation logic
8 }
```

Специализация от стандартната библиотека: `std::vector<bool>` е специализация на `std::vector<T>` – вместо масив от `bool` стойности, елементите се пакетират побитово (**bitset оптимизация по памет:** 1 бит на елемент вместо 1 байт). Заради тази оптимизация обаче `operator[]` не връща `bool&`, а проху обект, което нарушава стандартните очаквания за контейнер и е считано за проектна грешка.

4.5 Ограничение: дефиниции в хедър файл

Шаблоните не могат да се разделят на `.h` (декларация) и `.cpp` (дефиниция) по традиционния начин. При инстанциране компилаторът трябва да вижда цялото тяло на шаблона. Ако дефиницията е в отделен `.cpp`, `linker`-ът ще съобщи за липсващ символ. **Стандартна практика:** цялата имплементация в хедър файла (`.h`).

5 Множествено наследяване

C++ позволява произведен клас да има **повече от един директен базов клас**:

```
1 class Flyable {
2 public:
3     virtual void fly() const { std::cout << "Flying\n"; }
4 };
5
6 class Swimmable {
7 public:
8     virtual void swim() const { std::cout << "Swimming\n"; }
9 };
10
11 class Duck : public Flyable, public Swimmable {
12 public:
13     void quack() const { std::cout << "Quack!\n"; }
14 };
15
16 int main() {
17     Duck d;
18     d.fly();    // OK
19     d.swim();   // OK
20     d.quack();  // OK
21 }
```

Предимства: комбиниране на независими интерфейси/функционалности; повторна употреба на код.

5.1 Диамантен проблем (Diamond Problem)

Проблемът възниква, когато един базов клас се наследява по **два независими пътя**, което води до **две отделни копия** на базовия клас в производния.

```
1 class Base { public: int x; };
2 class A : public Base { }; // A::Base::x
3 class B : public Base { }; // B::Base::x -- separate copy!
4 class C : public A, public B { };
5
6 int main() {
7     C obj;
8     // obj.x = 5; // ERROR: ambiguous -- A::x or B::x ?
9     obj.A::x = 5; // OK: explicit disambiguation
10    obj.B::x = 10; // OK
11 }
```

Конструкторът на Base се извиква **два пъти** – веднъж чрез A и веднъж чрез B.

5.2 Виртуално наследяване – решение на диамантения проблем

Ключовата дума `virtual` пред базовия клас гарантира, че в крайния наследник ще съществува **само едно споделено копие** на общия базов клас.

```
1 class Base { public: int x; };
2 class A : virtual public Base { }; // virtual inheritance
3 class B : virtual public Base { }; // virtual inheritance
4 class C : public A, public B { }; // one shared Base::x
5
6 int main() {
7     C obj;
8     obj.x = 42; // OK: no ambiguity, only one Base
9 }
```

5.3 Конструктори при виртуално наследяване

При виртуално наследяване **само най-производният клас** (в случая C) инициализира виртуалния базов клас. Инициализациите на Base в A и B се **игнорират**:

```
1 class Base {
2 public:
3     Base(int v) { std::cout << "Base(" << v << ")\n"; }
4 };
5
6 class A : virtual public Base {
7 public:
8     A() : Base(0) { } // Base(0) is IGNORED when constructing through C
9 };
10
```

```

11 class B : virtual public Base {
12 public:
13     B() : Base(1) { } // Base(1) is IGNORED when constructing through C
14 };
15
16 class C : public A, public B {
17 public:
18     C() : Base(42), A(), B() { } // Base(42) is the one that runs
19 };
20 // Output: Base(42)  A()  B()  C()

```

Ред на конструиране при виртуално наследяване:

1. Конструктори на виртуалните базови класове (извикани от най-производния клас).
2. Конструктори на неvirtуалните базови класове (в реда на декларирането им).
3. Конструктори на член-данните (в реда на декларирането им).
4. Тялото на конструктора на самия клас.